

example, your object may want to close a file, close a network connection, and so on.

When an object is deleted, Python checks to see if any of its instance variables refer to other objects. If so, the reference counts of those objects are also decremented. If this results in another object's reference count going to zero, then that object is deleted as well. This type of chained or *cascading* deletion can go on as many layers deep as is necessary. [Listing 11-2](#) provides an example.

File: DeleteExample_Teacher_Student.py

```
# Student class

class Student():
    def __init__(self, name):
        self.name = name
        print('Creating Student object', self.name)

    ❶ def __del__(self):
        print('In the __del__ method for student:', self.name)

# Teacher class
class Teacher():
    def __init__(self):
        print('Creating the Teacher object')
        ❷ self.oStudent1 = Student('Joe')
        self.oStudent2 = Student('Sue')
        self.oStudent3 = Student('Chris')

    ❸ def __del__(self):
        print('In the __del__ method for Teacher')

# Instantiate the Teacher object (that creates Student objects)
    ❹ oTeacher = Teacher()

# Delete the Teacher object
    ❺ del oTeacher
```

Listing 11-2: Classes demonstrating `__del__()` methods